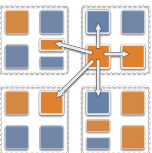


Reconverse: A new communication layer for task-based runtime systems

Ritvik Rao

Parallel Programming Laboratory
University of Illinois at Urbana-Champaign



Charm++ software stack

Application

Charm RTS:
Objects, Location Management, Load balancing

Per-collection QD

Location
management

Object groups and pe/node level abstractions

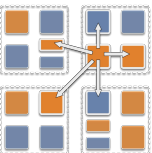
Global QD (Quiescence Detection)

Converse:
Scheduling, Queueing, ULTs

LRTS
Startup, Communication

Motivation

- Existing converse communication layer: Old and complicated
 - Many lines of code from various machine layers
 - Features that are deprecated-nonexistent, but code exists
- Difficult to maintain
 - Need to consider entire Charm software stack
 - Adding/removing machine layers is very difficult
- Converse tightly coupled to Charm++
 - Cannot write another runtime on top of Converse

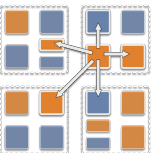


Reconverse

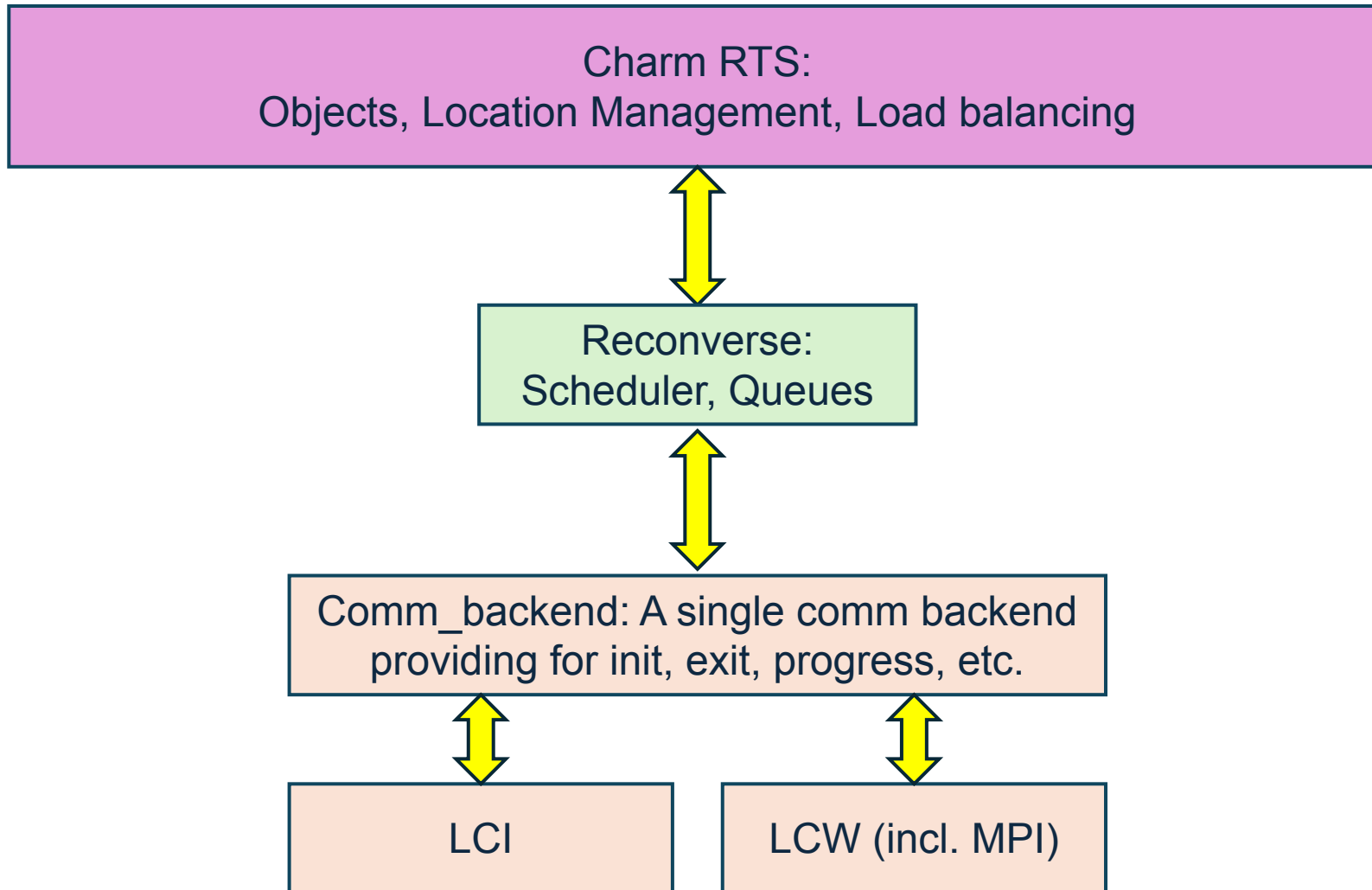
- New communication layer for Charm++ and other asynchronous message-passing runtime systems
- Implemented as separate code base
- Built on LCI, a new communication library for asynchronous multithreaded runtimes
- Includes experimental changes not present in old

Reconverse

- Queue registration
- Decrement-and-enqueue

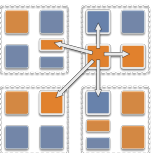


Structure



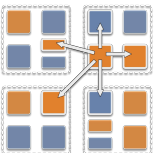
Communication layers

- **LCI: Lightweight Communication Interface**
 - Fixes MPI shortcomings in multithreaded performance
 - Atomic objects (lock-free) for completion objects
 - User-controlled network progress polling
 - Adjustable by command-line parameters in Reconverse
 - Can choose frequency/threads that do polling
- **LCW: Communication wrapper**
 - Common wrapper for multiple communication libraries
 - Currently supports MPI
 - Possible to add other backends via LCW



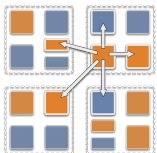
Communication backend

- Interfaces with LCI and LCW
- Simple backend with commands needed for a message passing runtime
 - init and exit
 - barrier
 - progress
- Significantly smaller than old Converse implementation



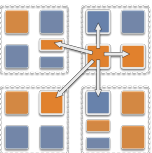
Reconverse Model

- PEs (processing elements) = threads
 - `std::thread` (compared to `pthread` in old converse)
 - 1 thread per CPU core
 - Each thread has state (PE number, rank number)
 - 1 or more threads per process
- Each thread contains its own scheduler
 - Different from other programming models with master thread and worker threads
- Unlike converse, no dedicated communication thread
 - Atomics in LCI allow for shared network polling among threads



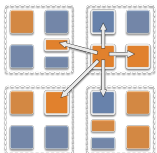
Reconverse scheduler and queues

- After startup, each thread enters a scheduler loop
- Multiple queues, each containing data and handlers that trigger specific C++ functions registered at startup
- Currently available queues
 - MPSC fifo queue for Converse-level PE/thread messages
 - MPMC fifo queue for Converse-level process messages
 - SPSC priority queue for Charm++-level PE/thread messages
 - MPMC priority queue for Charm++-level node messages
 - Progress queue (provided by LCI)



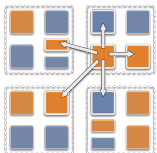
New to Reconverse: Queue registration

- Usage of queues depends on application
 - For example, some may rely on node queues disproportionately
- Each scheduler loop checks a queue, so we want to maximize probability of work being performed
- Therefore, allow for queue registration
 - Select which queues are polled, and how often they are polled
 - Initial implementation: set by user before run time
 - Later: allow for queue polling frequencies to be dynamically changed by application based on success rate



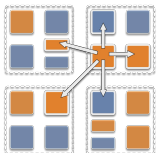
Queue registration example

- Say I want to poll PE queue 30% of the time, node queue 5%, local priority queue 35%, node priority queue 5%, and network progress 25%
- Step 1: Register queues in some order by adding polling function
 - `addQueue(pollConverseThreadQueue)`
 - `addQueue(pollConverseNodeQueue)`
 - etc.
- Step 2: Set frequency
 - `setFrequency([30,5,35,5,25])`
- Reconverse normalizes frequency weights out of 64 slots, and assigns each polling function to a slot
- Each scheduler loop then combines loop counter with a mask to call functions



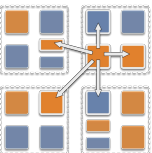
Other configurable options in reconverse

- +lci_ndevices: Number of LCI device abstractions to be set per process
 - LCI devices map to real network devices
- +backend_poll_freq: How frequently should a thread poll network progress
 - E.g. if this is set to 4, threads poll progress once every 4 scheduler loops
- +backend_poll_thread: Which threads should actually call progress
 - If $\text{CmiMyRank()} \% \text{backend_poll_thread} == 0$, poll



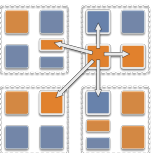
Challenges

- **Backward compatibility**
 - Need to adopt features used by flagship applications
 - examples:
 - NAMD and MPI-interoperable Charm++ use user-driven scheduling
 - Existing job scripts have references to deprecated features (i.e. commthreads and commap)
- **How to deal with features tightly coupled with Charm++?**
 - example: application tracing and registration in Converse
- **Performance debugging**
 - Matching application performance with benchmark performance



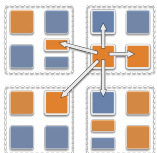
Progress

- Feature-complete implementation that now can run NAMD and ChaNGa correctly
- Implementation of registered queues
- RDMA communication on both CPUs and GPUs

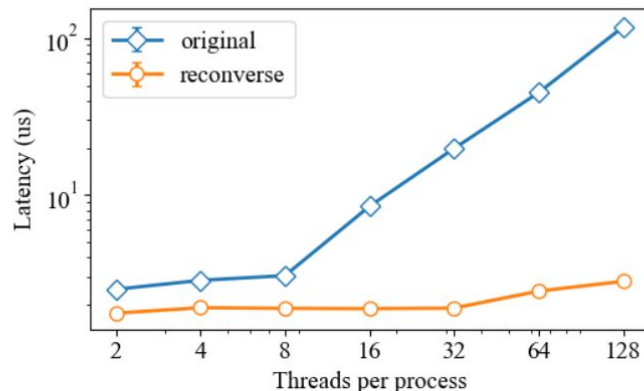


Experiments

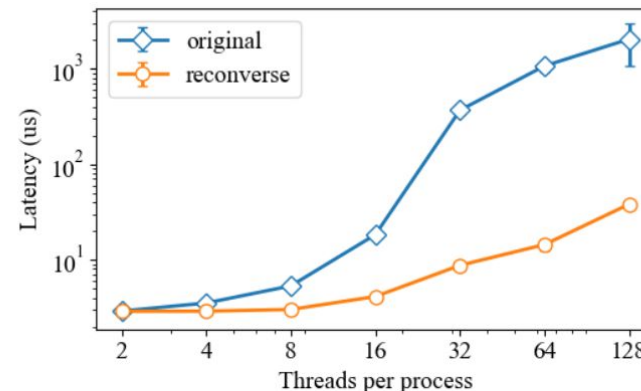
- Benchmarks, mini-apps, larger apps
- Experiments on benchmarks and mini-apps from Jiakun on NERSC Perlmutter and Purdue Anvil
- ChaNGa results from me (Ritvik) on NCSA Delta and Anvil
- NAMD results from Eric Bohm on TACC Vista (Grace-Grace nodes)
- Machine configurations:
 - Old converse on Anvil and Vista: UCX layer
 - Old converse on Perlmutter and Delta: OFI layer



Benchmark Data: Pingpong

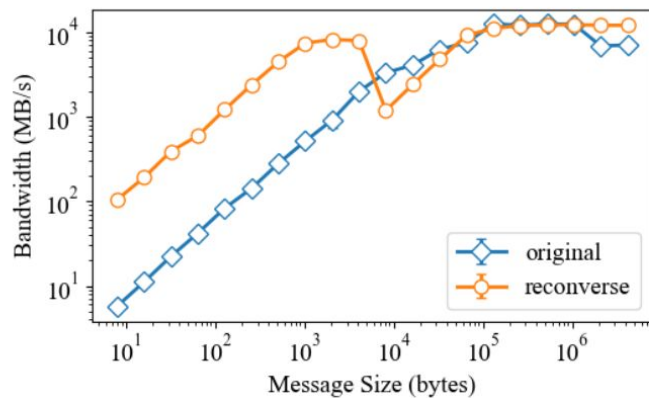


(a) Pingpong latency results on Anvil.

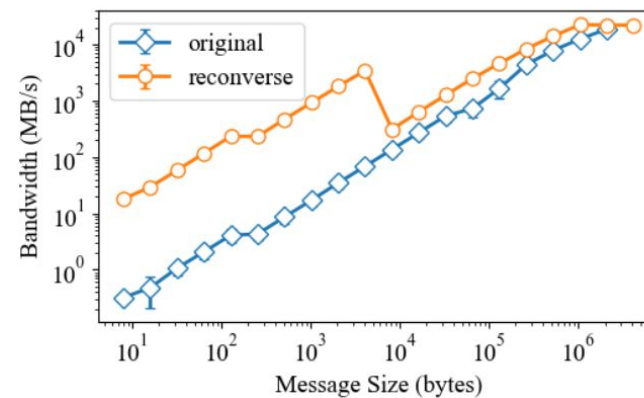


(b) Pingpong latency results on Perlmutter.

Figure 6.8: Pingpong latency results on Anvil and Perlmutter. All messages are 8-byte. The number of threads per process varies from 2 to 128.



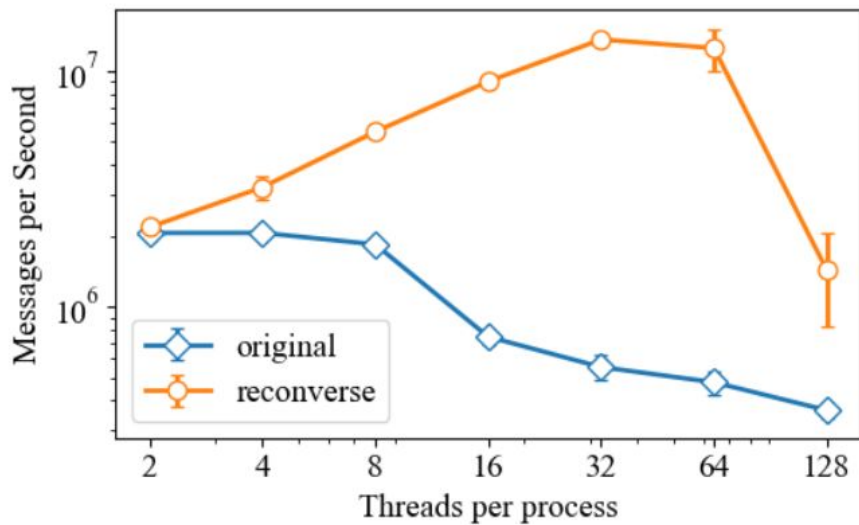
(a) Pingpong bandwidth results on Anvil.



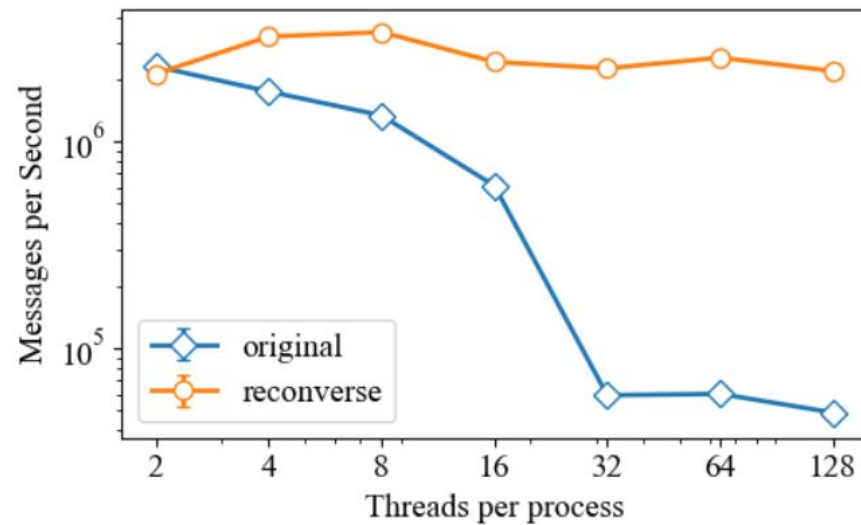
(b) Pingpong bandwidth results on Perlmutter.

Figure 6.9: Pingpong bandwidth results on Anvil and Perlmutter. We use 64 PE pairs with message sizes varying from 8 bytes to 4 MiB.

Benchmark Data: Ping-Ack

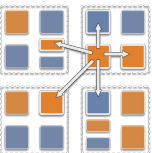


(a) Pingack Message Rate (56B) on Anvil.



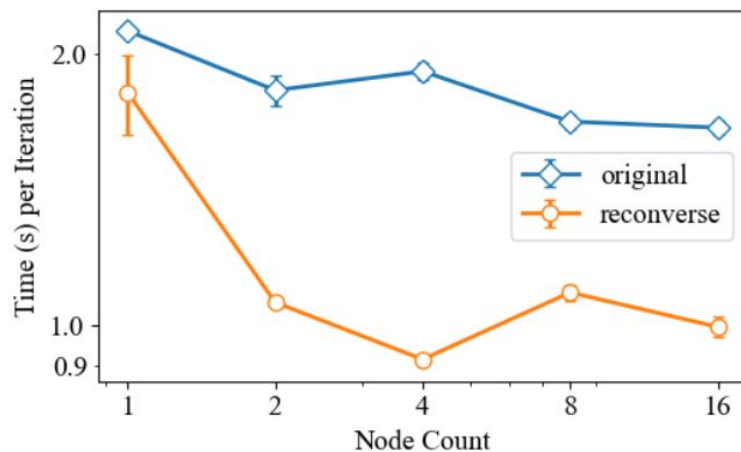
(b) Pingack Message Rate (56B) on Perlmutter.

Figure 6.10: Pingack Message Rate with 56-byte messages on Anvil and Perlmutter. Threads number per process varies from 2 to 128.

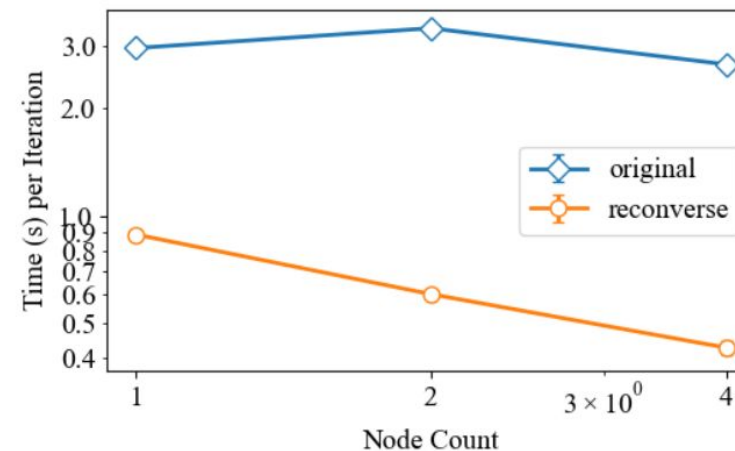


Application data: Barnes-Hut

- Does Barnes-hut n-body simulation



(a) Barnes-Hut average time per iteration on Anvil.

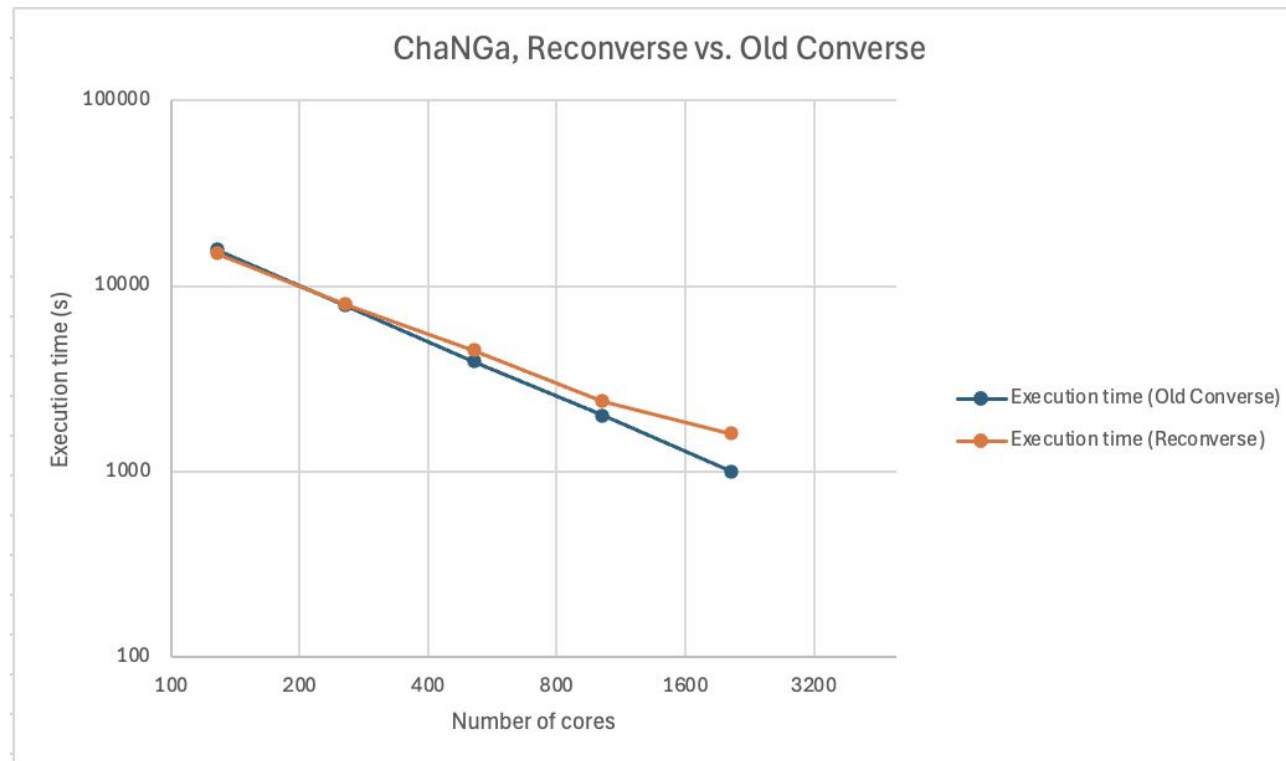


(b) Barnes-Hut average time per iteration on Perlmutter.

Figure 6.13: Barnes-Hut average time per iteration on Anvil and Perlmutter with different number of compute nodes.

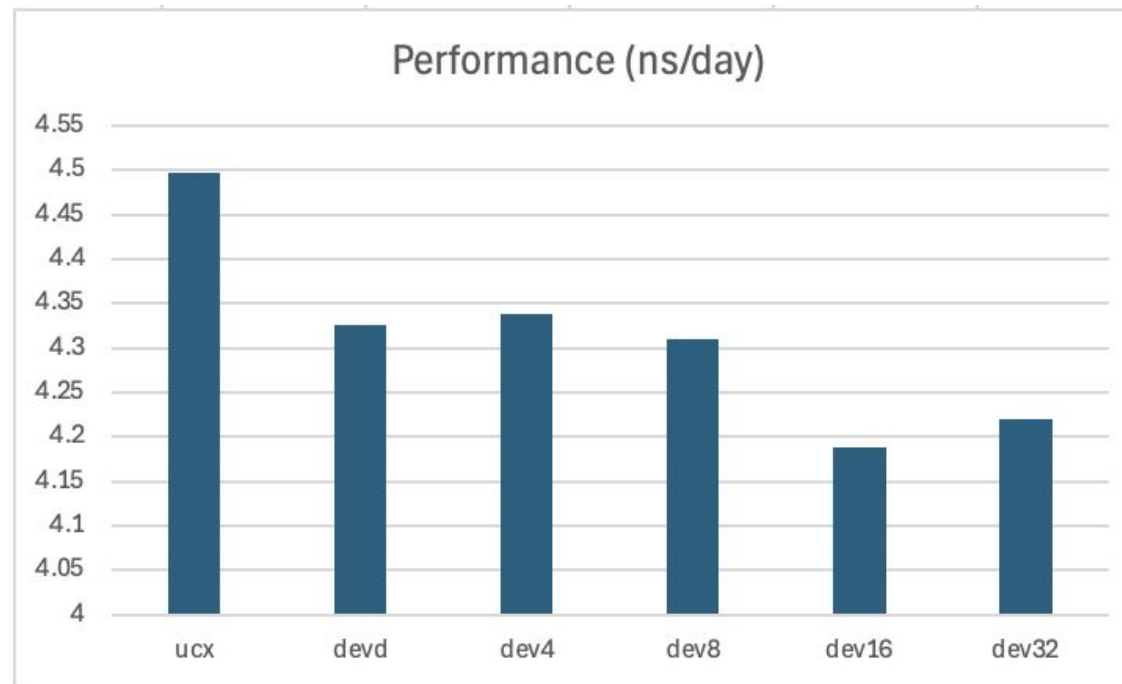
Application Data: ChaNGa

- ChaNGa = Charm N-Body Gravity, developed in collaboration with N-Body shop at UW
- Strong scaling, 2B particles, CPU-only, Anvil



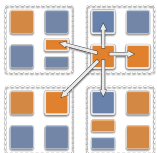
Application data: NAMD

- Nanoscale Molecular Dynamics, developed with Theoretical and Computational Biophysics group at UIUC
- Running on 1 node on TACC Vista



Analysis of results

- Speedups on benchmarks are large
 - Most of benefit is from LCI (lock-free structures, flexible network polling)
 - Some benefit is from a more efficient scheduler loop (fewer queue checks, method calls, etc)
 - Delta/Perlmutter performance is better due to poorly optimized OFI layer in old Converse
- But performance isn't translating yet to larger applications
 - Aside from smaller runs on Delta (due to flawed OFI), performance is worse for NAMD and ChaNGA despite tuning
 - We suspect the main culprit is the lack of pinned memory registration, important for coarse-grained messages
 - Load balancing also appears to be broken, with high idle times



Next steps

- Fix lingering bugs
 - Some build configurations don't work
 - Error when many threads enqueue to the same Converse queue
- Need to create new benchmarks to test new features
 - e.g. What benchmarks can be used to test queue registration?
- Merge reconverse build into Charm++ main branch
 - We are going to move away from old converse and rely entirely on reconverse
- Promote reconverse as a layer that can be used by other task-based runtimes



Discussion

- Modularization for software sustainability
- Can you use a single comm layer for many runtimes?
- Usefulness of queue reg, etc.

